

产品报告

LSD

2025 年 12 月 15 日

目录

1	产品功能介绍	1
2	程序概要设计	6
2.1	数据模型层	6
2.2	服务层	7
2.3	界面层	14
2.4	服务端	14
3	软件架构	19
4	技术亮点及实现原理	23
5	开发日志:	25

1 产品功能介绍

账号体系

支持向服务端注册与登录，登录后自动持久化会话，即重启应用可直接进入主界面。



(a) 登录界面

(b) 注册界面

图 1: 聊天应用用户认证界面

通讯录

登录后从服务端批量拉取（更新或插入）联系人并写入本地数据库，保障联系人列表与服务器一致，以字典序按条目显示联系人基本信息。

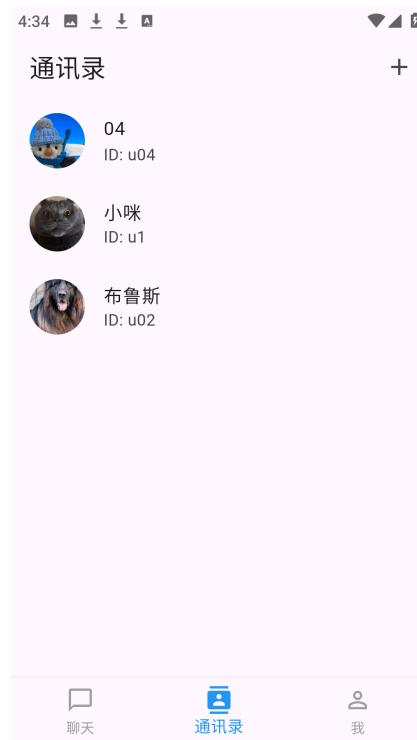
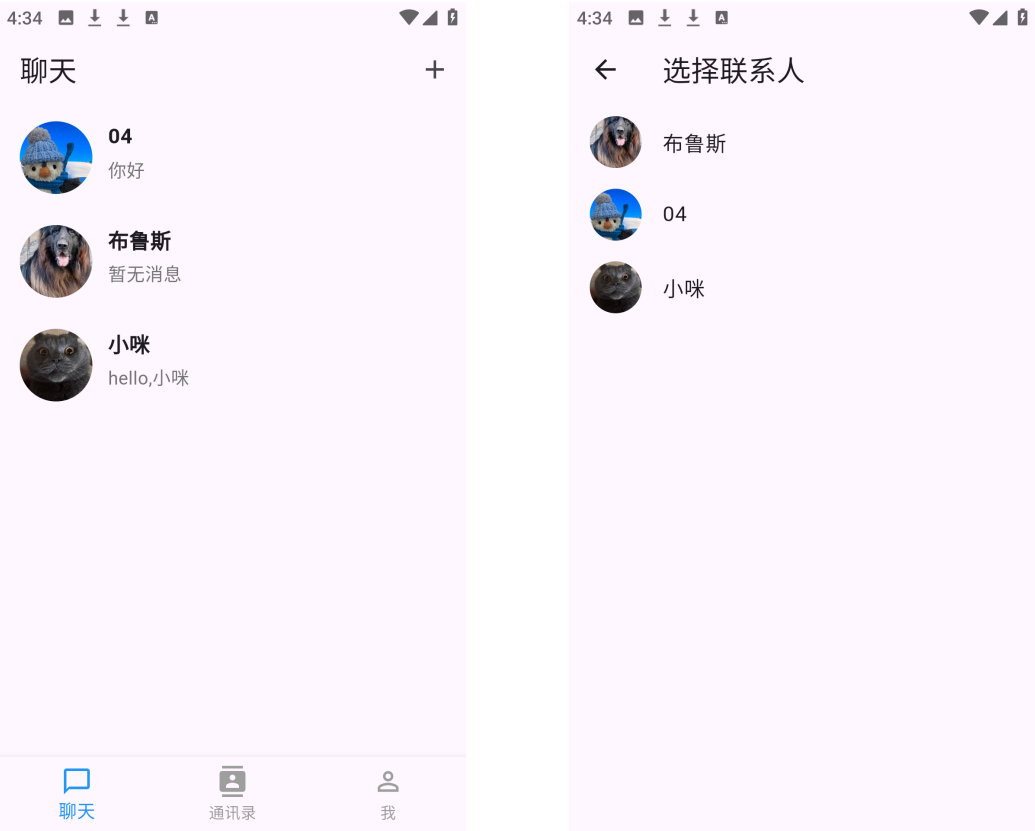


图 2: 通讯录

聊天会话管理

点击右上角”+” 按钮，可拉取联系人信息，从而创建一对一聊天会话。
向左拨动聊天条目，右端显示红色删除按钮，点击可删除该对话。



(a) 聊天列表

(b) 新建列表

图 3: 聊天列表相关

个人资料管理

个人中心，支持更新头像与用户名，并将变更同步到服务器及本地缓存，从而在自己以及联系人的列表正确刷新与显示。



图 4: 个人中心

消息收发

允许实时消息推送，可发送三种消息，文本，公共资源表情，文件（图片）。

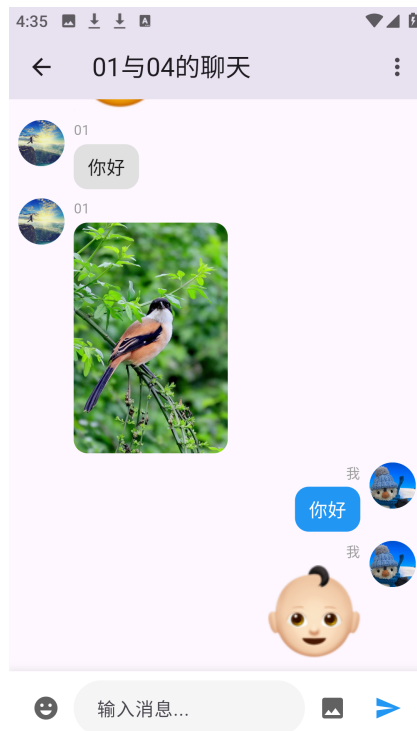


图 5: 聊天界面

2 程序概要设计

2.1 数据模型层

models 下定义 User、Chat、Message 等实体，负责描述数据结构与序列化逻辑。

User

- final String id; 用户唯一 uid，注册是自行申请
- final String? name; 用户名称，允许随意修改
- final String? avatar; 用户头像路径，本地版本下为文件路径，后改为 CS 模式后该条为网络 url
- final String? passwordHash; 用户密码，本地模式下用于用户校验，后弃用
- final String? token; 服务器验证令牌，用户登陆后服务器将令牌传给用户，此后用于用户向服务器发送消息以及服务器解析用户

Chat

- final String id; 对话唯一 uid，通过时间戳生成保证唯一
- final String name; 对话名称，在服务端统一形如 a 与 b 的聊天，其中 a 为主动申请构建该对话的用户。后聊天列表为了避免困惑，将聊天对话的头像和昵称均更改为显示为聊天另一方的信息
- final List<String> participantIds; 参与该对话的用户的 id，序列化为使用逗号拼接。对于一对一对话这样的设计不如直接使用两个 string，数据库查询方便。这有利于拓展参与用户数量，比方说建立群聊
- final String? lastMessage; 最后一条信息，用于聊天列表的显示
- final DateTime? lastMessageTime; 最后一条消息的时间

- final int unreadCount; 未读信息数，作为拓展接口，我们实际上并没有做相关逻辑

Message

- final String id; 消息唯一 id，由发送用户 id，发送时间等信息拼接而成
- final String chatId; 该消息所隶属对话的 id
- final String senderId; 发送该消息的用户，可集成在消息 id 中
- final String senderName; 该条冗余，使用 senderId 即可
- final MessageType type; 消息类型，允许发送三种枚举类型，text,image,emoji
- final String content; 消息内容，对应为文本信息或是网络 url 或是本地文件路径
- final DateTime timestamp; 消息发送时间
- final bool isRead; 未作具体实现

2.2 服务层

services 下定义若干服务，用于具体的业务实现逻辑

AuthService

AuthService 负责应用的用户认证与会话管理，是客户端与服务器进行身份交互的核心模块。该服务通过调用服务器提供的注册与登录接口完成用户身份验证，并在登录成功后维护当前用户的基本信息及访问令牌(Token)。同时，AuthService 负责将登录状态持久化到本地，以支持应用重启后的会话恢复，并在用户登出时统一清理内存状态与本地缓存。此外，该服务在用户登录后触发联系人与聊天数据的同步，为后续业务模块提供可靠的身份与权限基础。

- AuthService.instance 单例对象，全局唯一的认证服务实例，用于统一管理登录态与用户会话。
- init() 初始化认证状态，从 SharedPreferences 中读取本地缓存的用户信息，若存在有效数据则恢复当前登录用户并通知监听者。
- registerUser(String uid, String name, String password) 客户端—服务器模式下的用户注册接口，通过 HTTP POST 调用服务器注册 API，由服务器负责密码哈希与合法性校验，本地不落库，仅在成功后返回结果。
- login(String username, String password) 用户登录函数，通过调用服务器登录 API 验证身份，登录成功后获取 token 并构造当前用户对象，同时将用户信息写入本地数据库并触发联系人与聊天数据同步。
- _syncFromServer() 登录成功后从服务端数据库拉取数据到本地缓存，包括与当前用户关联的联系人 contacts 表，对应的 users 表，关联的对话 chats 表，本地缺失的 messages 等等
- logout() 用户登出操作，清空当前内存中的用户信息，并移除本地 SharedPreferences 中的登录缓存数据。
- updateAvatar(File? file) 更新用户头像，当传入文件时通过 multipart 请求上传至服务器，服务器返回头像 URL 后同步更新本地数据库与当前用户状态。
- updateName(String name) 更新用户名信息，先向服务器提交修改请求，成功后再更新本地数据库与缓存，保证本地状态与服务器数据一致。
- initials() 获取当前用户名首字母（大写），用于无头像情况下的用户占位展示。

ChatService

ChatService 是聊天业务逻辑的核心服务，负责聊天会话管理、消息收发以及实时通信处理。该服务在用户登录后从本地数据库加载会话与历史消息，并通过服务器事件推送机制（SSE）实时接收新消息。ChatService 统一处理文本、图片与表情等不同类型的消息，完成消息去重、本地落库、内存缓存更新以及会话摘要刷新等操作。通过将网络通信、数据同步与 UI 状态管理集中在同一服务中，ChatService 实现了聊天功能的高内聚与可维护性。

- ChatService(AuthService authService) 聊天服务构造函数，依赖认证服务获取当前用户信息与访问令牌，统一负责聊天会话、消息收发及实时通信逻辑。
- initialize() 初始化聊天服务，在应用启动或用户进入聊天模块时调用，从本地数据库加载聊天会话列表，为后续消息展示做准备。
- onLogin() 用户登录成功后调用，清理旧的聊天状态并重新加载本地聊天数据，同时建立服务器事件推送（SSE）连接以接收实时消息。
- onLogout() 用户登出时调用，关闭 SSE 连接并清空内存中的聊天与消息缓存，恢复初始状态。
- loadChats() 从本地数据库中加载当前用户参与的所有聊天会话，并同步到内存列表中供界面展示。
- chats 当前内存中的聊天会话列表（只读），作为会话列表界面的数据来源。
- messagesOf(String chatId) 根据聊天 ID 返回对应的消息缓存列表，若尚未加载则返回空列表。
- loadMessagesIfNeeded(String chatId) 首次进入某个聊天页面时调用，从本地数据库加载该会话的历史消息并缓存到内存中，避免重复加载。

- `loadMessages(String chatId, int limit, int offset)` 分页加载指定聊天的历史消息，支持向上翻页查看更早的聊天记录。
- `startSseListener(String? chatId)` 启动服务器推送监听（SSE），用于实时接收服务器推送的消息事件，支持全局监听或指定会话监听。
- `stopSseListener()` 关闭当前 SSE 连接并清理相关资源，防止重复连接或内存泄漏。
- `_handleIncomingMessage(Map<String, dynamic> data)` 处理服务器通过 SSE 推送的单条消息数据，完成消息去重、本地数据库写入、缓存更新及会话摘要刷新。
- `createChat(String chatName, List<String> participantIds)` 创建新的聊天会话，当前仅支持一对一聊天，会先调用服务器接口创建会话，再同步写入本地数据库。
- `deleteChat(String chatId)` 删除指定聊天会话，调用服务器接口完成删除后，同时清理本地数据库中的会话及相关消息数据。
- `sendTextMessage(String chatId, String senderId, String senderName, String text)` 发送文本消息，通过服务器接口完成消息发送，消息的最终存储与推送由服务器统一处理。
- `sendImageMessage(String chatId, String senderId, String senderName)` 发送图片消息，用户从本地相册选择图片后先上传文件至服务器，再以图片路径作为消息内容发送。
- `sendEmojiMessage(String chatId, String senderId, String senderName, String emojiPath)` 发送表情消息，表情资源为预置内容，作为特殊类型消息发送给服务器。
- `markAsRead(String messageId)` 将指定消息标记为已读状态，更新本地数据库中的消息读取标记，用于未读数统计。
- `dispose()` 销毁聊天服务实例时调用，确保关闭 SSE 连接并释放相关资源。

DatabaseService

DatabaseService 负责本地 SQLite 数据库的初始化与访问，是应用的数据持久化层。该服务统一管理用户、聊天会话、消息及联系人等核心数据表的创建与维护，并对外提供标准的增删改查接口，以支持聊天记录分页加载、会话排序以及联系人查询等功能。在客户端—服务器模式下，DatabaseService 还承担服务器数据同步后的本地覆盖写入职责，确保本地缓存与服务器状态保持一致，从而提升应用的响应速度与离线可用性。

- DatabaseService.instance 数据库服务单例对象，负责统一管理本地 SQLite 数据库的创建、访问与关闭，确保全局仅存在一个数据库实例。
- database 获取数据库实例的方法，采用惰性初始化方式，在首次访问时创建并打开数据库连接。
- getUserById(String userId) 根据用户唯一 ID 从本地 users 表中查询用户信息，返回对应的用户对象或空值。
- __initDB(String filePath) 初始化本地数据库文件，配置数据库路径、版本号及外键等基础属性。
- __createDB(Database db, int version) 数据库首次创建时执行，负责创建用户表、聊天会话表、消息表以及联系人表等核心数据结构。
- insertTestUsers(Database db) 在注册逻辑未完成前插入测试用户数据，仅用于开发和调试阶段。
- getUserAvatar(String userId) 根据用户 ID 查询其头像信息，若头像不存在或为空则返回空值。
- insertChat(Chat chat) 插入或更新聊天会话数据，当会话已存在时自动覆盖旧记录，保证本地会话唯一性。
- deleteChat(String chatId) 根据聊天 ID 删除本地聊天会话记录，返回受影响的行数。

- `getAllChats()` 从本地数据库中读取所有聊天会话，按最后一条消息时间倒序排列，用于会话列表展示。
- `updateChatLastMessage(String chatId, String lastMessage, DateTime lastMessageTime)` 更新指定聊天会话的最后一条消息内容及时间，用于刷新会话列表中的摘要信息。
- `insertMessage(Message message)` 将单条聊天消息写入本地消息表，若消息已存在则覆盖原记录。
- `insertMessages(List<Map<String, dynamic>> maps)` 批量插入多条消息数据，使用事务与批处理机制提升同步性能，并自动忽略重复消息。
- `getMessagesByChat(String chatId, int limit, int offset)` 按分页方式获取指定聊天的消息记录，用于历史消息加载与聊天界面滚动展示。
- `markMessageAsRead(String messageId)` 将指定消息标记为已读状态，用于未读消息统计与状态更新。
- `getLastMessageTime(String chatId)` 查询指定聊天会话的最后一条消息时间戳，用于消息同步与增量拉取逻辑。
- `getAllContacts(String currentUserId)` 查询当前用户的所有好友列表，通过 `contacts` 表关联 `users` 表返回好友的基本信息。
- `addContact(String targetUserId)` 添加好友操作，先调用服务器接口建立好友关系，成功后将联系人关系及好友信息写入本地数据库。
- `overwriteContacts(List<Map<String, dynamic>> contacts)` 覆盖写入联系人数据，在登录或同步时完全清空旧联系人表并写入服务器最新数据。
- `overwriteChats(List<Map<String, dynamic>> chats)` 覆盖写入聊天会话数据，用于服务器同步场景下以服务端数据为准重建本地会话表。
- `syncContactsFromServer(AuthService authService)` 从服务器拉取联系人列表并同步至本地数据库，采用全量覆盖方式保证本地数据与服务器一致。

- `close()` 关闭数据库连接，在应用退出或资源释放时调用。

FileService

FileService 负责图片与表情等文件资源的本地存储与管理，是聊天多媒体功能的基础支持模块。该服务统一维护应用内部的文件目录结构，提供图片保存、删除、大小查询以及过期文件清理等功能。同时，在应用初始化阶段，FileService 会将内置表情资源从 `assets` 拷贝至本地存储，以提升访问效率。在去除 `image_file` 表以后，FileService 实际上只涉及到对于表情资源的访问

- `FileService.instance` 文件存储服务的单例对象，统一负责应用中图片与表情资源的本地存储与管理。
- `imageDir` 获取并维护图片存储目录，若目录不存在则自动创建，用于保存聊天过程中产生的图片文件。
- `emojiDir` 获取并维护表情资源存储目录，用于缓存内置表情文件，避免频繁从 `assets` 中读取。
- `saveImage(File imageFile, String fileName)` 将指定图片文件保存至本地图片目录，返回保存后的文件路径，用于消息内容或数据库记录。
- `deleteImage(String filePath)` 删除指定路径的图片文件，用于消息撤回或缓存清理等场景。
- `initEmojis()` 初始化本地表情资源，将 `assets/emojis` 目录下的表情图片复制到本地存储中，并返回所有已加载表情文件的本地路径列表。
- `getFileSize(String filePath)` 获取指定文件的大小信息，用于图片消息的元数据统计或展示。
- `cleanOldFiles()` 清理本地图片缓存，自动删除存储时间超过一定期限（如 30 天）的图片文件，以控制本地存储空间占用。

2.3 界面层

screens 目录下包含程序用到的全部界面构建组件，共包含八个界面，分别是聊天列表界面，聊天界面，联系人界面，底部导航界面，登录界面，个人主页界面，注册界面，选择联系人界面。

widgets 提供导航栏、消息气泡、表情面板等复用组件。

2.4 服务端

服务端提供服务的路由接口主要定义在 `userHandler.cpp` 中，包括四条 http 路由。

register__user__routes

这条路由用于管理与用户相关的一些请求，主要涉及到服务端的 `users` 表。

- `Post"/api/register"` 注册逻辑
- `Post"/api/login"` 登录逻辑，登录成功后返回给用户 token
- `Get"/api/users/batch"` 批量查询用户信息，用于本地数据库同步联系人数据
- `Put"/api/users/name"` 更改用户名，保存至服务器数据库，并将新数据返回
- `Post"/api/users/avatar"` 更改头像，与用户名的更改类似，但是需要上传头像文件到服务端

register__contacts__route

这条路由用于管理与联系人相关的一些请求，主要涉及到服务端的 `contacts` 表。

- `Get"/api/contacts"` 获取当前用户的所有联系人
- `Post"/api/contacts/add"` 添加一个新的联系人

register__chats__route

这条路由用于管理与对话相关的一些请求，主要涉及到服务端的 chats 表与 messages 表。

- Get”/api/chats” 批量查询涉及当前用户的 chats
- Post”/api/chats/add” 新增一条对话，不对该条目进行去重，因此允许两个用户间存在多个对话
- Delete R”(/api/chats/([A-Za-z0-9_]+))” 删除单条对话
- Post”/api/messages/send” 发送一条消息，并且广播 SSE
- Post”/api/files/upload” 文件上传，主要用于对话中图片文件
- Get”/api/messages” 拉取与该用户相关的消息，增量拉取

register__realtime__routes

这条订阅路由用于维护长连接，从而实时推送消息，实现实时对话

- Get”/api/stream” 创建 SSE 对话，并注册订阅

表 (1): chats

	id	name	rticipantId	lastMessage	lastMessageTime	unreadCount
	过滤	过滤	过滤	过滤	过滤	过滤
1	chat_1765467747295_41	不吃香菜与小味的聊天	u1,u2	在吗	1765607362100	0
2	chat_1765469871773_41	小味与cloud的聊天	u1,u3	assets/...	1765532023243	0
3	chat_1765469873911_41	小味与rabbit的聊天	u1,u4	[picture]	1765533297698	0
4	chat_1765545146859_41	不吃香菜与rabbit的...	u2,u4	发, 还会	1765560696850	0
5	chat_1765548097239_41	测试2.1与rabbit的...	u001,u4	不行	1765556926782	0
6	chat_1765548444564_41	测试2.1与臭味的聊天	u001,u3	yes	1765607191547	0
7	chat_1765548645306_1...	rabbit与臭味的聊天	u3,u4	好了吗	1765561518221	0
8	chat_1765557555744_41	小味与测试2.1的聊天	u001,u1	[picture]	1765561562184	0
9	chat_1765560976911_41	臭味与不吃香菜的聊天	u2,u3	[emoji]	1765561364215	0
10	chat_1765561582279_41	测试2.1与不吃香菜...	u001,u2	NULL	1765561582279	0
11	chat_1765564200633_1...	123与不吃香菜的聊天	u123,u2	NULL	1765564200633	0
12	chat_1765606043671_41	01与小味的聊天	u01,u1	hello,小味	1765606053880	0
13	chat_1765609680656_41	03与04的聊天	u03,u04	[picture]	1765613110679	0
14	chat_1765609831768_41	03与小味的聊天	u03,u1	你好	1765609834653	0
15	chat_1765610868883_41	03与布鲁斯的聊天	u02,u03	[picture]	1765611377899	0
16	chat_1765612926306_41	鸟与不吃香菜的聊天	u03,u2	nihao	1765612934395	0
17	chat_1765701259087_41	01与04的聊天	u01,u04	[emoji]	1765701325858	0

图 6: chats 表

表(T): contacts

	<u>id</u>	<i>contactId1</i>	<i>contactId2</i>	status	createdAt
	...	过滤	过滤	过滤	过滤
1	1	u1	u2	accepted	1765350613066
2	2	u3	u1	accepted	1765355561254
3	3	u3	u2	accepted	1765355565494
4	4	u4	u1	accepted	1765371882262
5	5	u4	u2	accepted	1765371885331
6	6	u4	u3	accepted	1765371888356
7	7	u001	u4	accepted	1765548094584
8	8	u001	u3	accepted	1765548426071
9	9	u1	u001	accepted	1765557552763
10	10	u001	u2	accepted	1765561578160
11	11	u123	u1	accepted	1765561984428
12	12	u123	u2	accepted	1765564198077
13	13	u01	u1	accepted	1765606041103
14	14	u02	u01	accepted	1765606227000
15	15	u02	u1	accepted	1765606245132
16	16	u02	u2	accepted	1765606251626
17	17	u03	u02	accepted	1765606460546

图 7: contacts 表

表(T): messages

	id	chatId	senderId	senderName	type	content	timestamp	isRea
	过滤	过滤	过滤	过滤	过滤	过滤	过滤	过滤
85	u03-1765610873...	chat_176561...	u03	03	0	你好	1765610872967	
86	u02-1765610901...	chat_176561...	u02	布鲁斯	0	你好	1765610900968	
87	u02-1765611332...	chat_176561...	u02	布鲁斯	0	你好	1765611331923	
88	u02-1765611343...	chat_176561...	u02	布鲁斯	2	assets/emojis/...	1765611342713	
89	u02-1765611349...	chat_176561...	u02	布鲁斯	2	assets/emojis/...	1765611349461	
90	u03-1765611378...	chat_176561...	u03	03	1	http://...	1765611377899	
91	u03-1765612934...	chat_176561...	u03	鸟	0	nihao	1765612934395	
92	u04-1765612958...	chat_176560...	u04	04	0	16.02	1765612958636	
93	u04-1765613004...	chat_176560...	u04	04	2	assets/emojis/...	1765613003882	
94	u04-1765613074...	chat_176560...	u04	04	1	http://...	1765613074642	
95	u03-1765613106...	chat_176560...	u03	鸟	0	123	1765613105806	
96	u03-1765613107...	chat_176560...	u03	鸟	2	assets/emojis/...	1765613107487	
97	u03-1765613110...	chat_176560...	u03	鸟	1	http://...	1765613110679	
98	u01-1765701269...	chat_176570...	u01	01	2	assets/emojis/...	1765701268571	
99	u01-1765701274...	chat_176570...	u01	01	0	你好	1765701273950	
100	u01-1765701309...	chat_176570...	u01	01	1	http://...	1765701308524	
101	u04-1765701322...	chat_176570...	u04	04	0	你好	1765701322439	

图 8: messages 表

表(T): users

	id	name	avatar	passwordHash	token
	过滤	过滤	过滤	过滤	过滤
1	u1	小咪	http://127.0.0.1:8080...	1	eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJ5b3V
2	u2	不吃香菜	http://127.0.0.1:8080...	2	eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJ5b3V
3	u3	臭味	http://127.0.0.1:8080...	3	eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJ5b3V
4	u4	rabbit	http://127.0.0.1:8080...	4	eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJ5b3V
5	u001	测试2.1	http://127.0.0.1:8080...	1	eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJ5b3V
6	u123	123	NULL	123	eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJ5b3V
7	u01	01	http://127.0.0.1:8080...	938db8c9f82c8c...	eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJ5b3V
8	u02	布鲁斯	http://127.0.0.1:8080...	a953f09a1b6b67...	eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJ5b3V
9	u03	鸟	http://127.0.0.1:8080...	0b8efa5a3bf104...	eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJ5b3V
10	u04	04	http://127.0.0.1:8080...	6cd5b6e51936a4...	eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJ5b3V
11	u05	05	NULL	c97550ce8213ef...	NULL
12	u100	100	NULL	ad57366865126e...	NULL

图 9: users 表

3 软件架构

数据库结构

本程序的数据库结构如下，本地缓存的数据库与服务端的数据库基本一致。在登录完成后，调用 `auth_service` 中的同步函数，将本地缓存的数据库中各个表同步为服务端的数据库中和当前登录用户相关的最新条目。考虑到本地数据库一般用户也不可以访问，以及一台设备可能登录多个用户，因此我们并不总是直接清空再拉取，而是做一些补充。

相比之下，服务端数据库 `users` 表中多了 `passwordHash` 字段用于存储经 `bcrypt` 加密后的密码，以及用户的 `token` 信息。本地 `users` 中自然无需存储密码，也无需保存 `token`，在登录成功后将 `token` 存在内存中即可。

另外，由于首先在本地实现，后面设计了服务端数据库，因此我们其实删除了最开始用于缓存图片的 `image_files` 表。在本地时，由于所有用户运行在同一设备，因此文件资源是在本地目录下的。当通过服务端通信以后，我们需要把文件资源上传，这里其实是上传到固定的服务器 `uploads` 文件中，通过 `url` 返回给消息，因此无需再做图片缓存。而这里其实是存在优化空间的，因为所有图片资源都从网络上读取，使得用户在离线时无法获取任何网络资源，因此将网络资源下载到本地缓存是实际生产中必要的，这里我们暂不实现。

架构说明

- 入口: `main.dart` 将 `AuthService` 与 `ChatService` 注入全局 `MultiProvider`，决定进入登录流或主界面。
- UI 层: 登录/注册页触发认证流程；联系人/会话列表页与聊天详情页读取本地缓存并通过 `Provider` 感知服务状态变化。
- 服务层: `AuthService` 负责会话管理、联系人与资料同步；`ChatService` 负责会话列表、消息发送与历史加载，并协同 `FileService` 完成媒体读取与持久化。

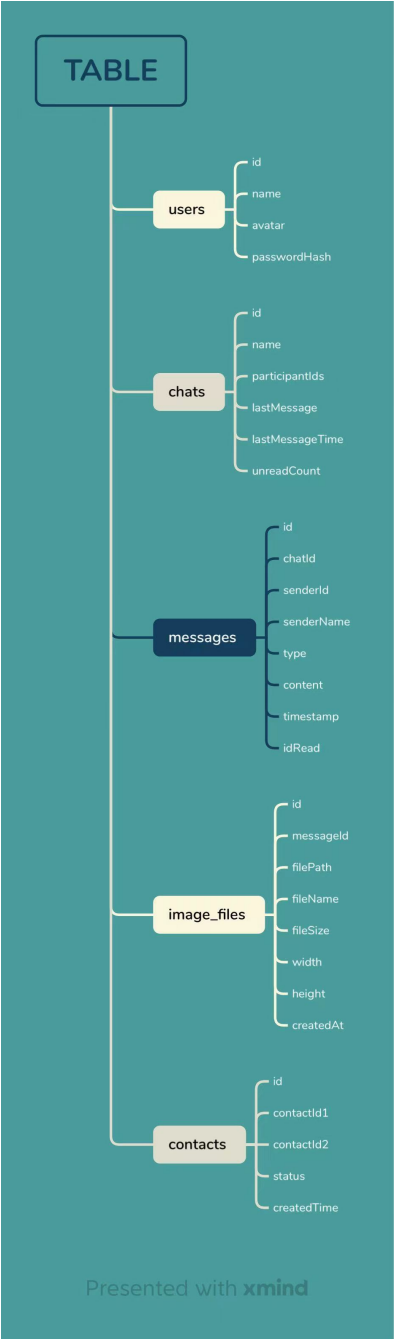


图 10: 聊天应用本地即服务器数据库表结构示意图

- 数据层: DatabaseService 封装 SQLite, 提供本地用户、联系人、聊天、消息的建表与 CRUD, 支撑 UI 的离线可用能力。
- 远端依赖: 通过 REST API 完成认证、联系人与聊天数据同步, 文件上传/下载由 'FileService' 对接独立存储端点。

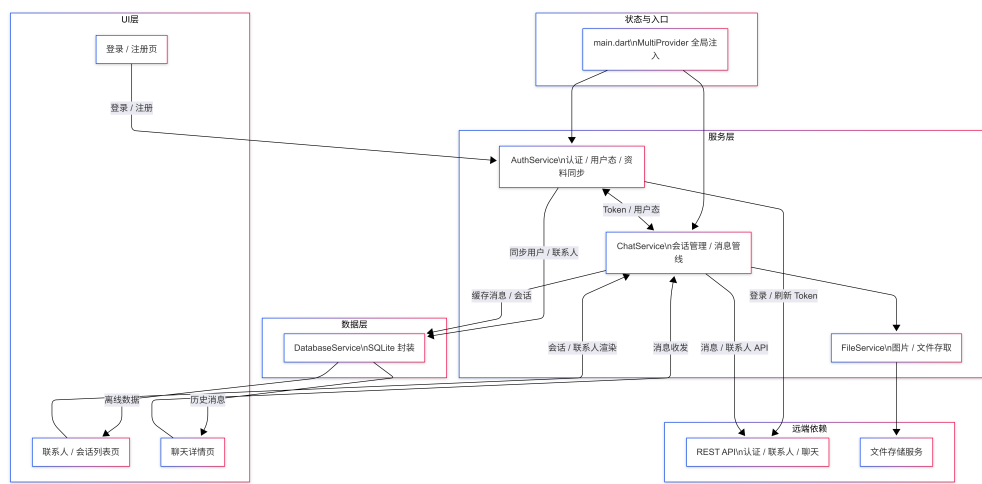


图 11: 软件架构图

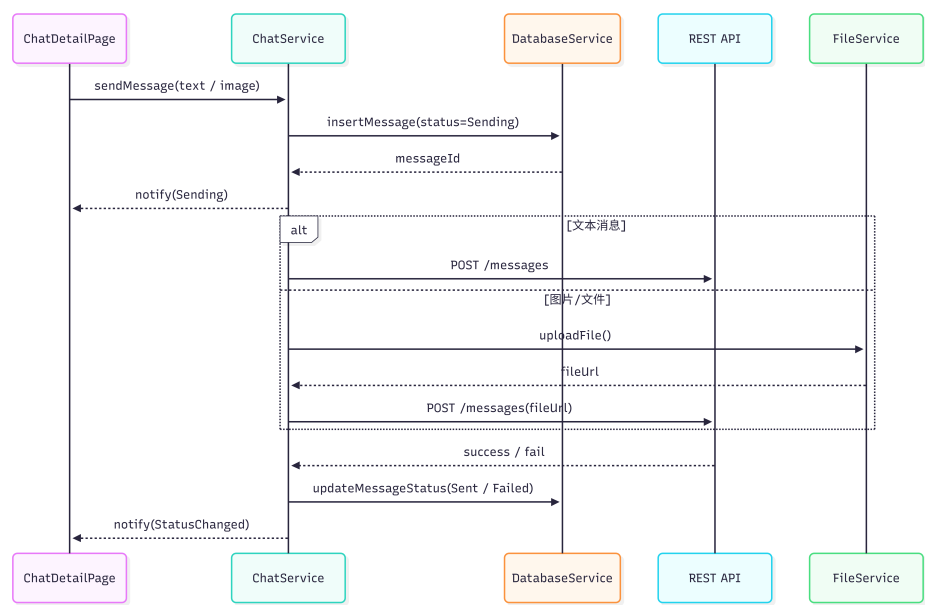


图 12: 发送消息时序图

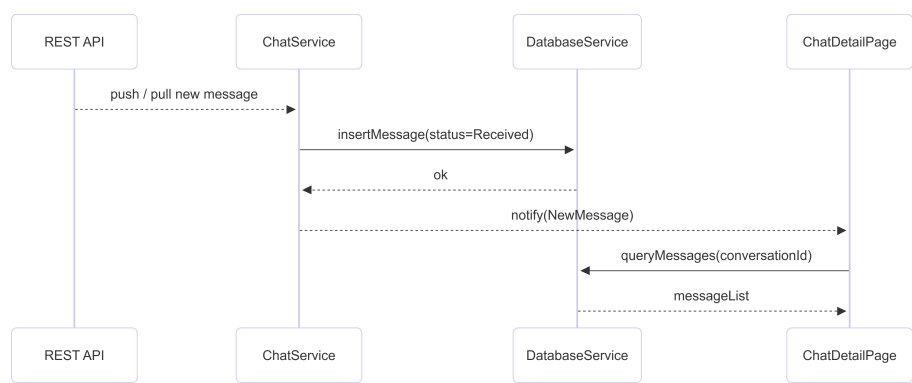


图 13: 接收消息时序图

4 技术亮点及实现原理

本地数据库与服务端的双向同步机制

系统采用“本地数据库优先”的设计思想，在用户登录成功后，由 `AuthService` 与 `ChatService` 协同完成联系人、会话及消息数据的初始化与持续同步。

在初始化阶段，客户端依次调用 `/api/contacts`、`/api/chats` 以及 `/api/messages` 接口，从服务端拉取最新的数据快照。其中，联系人与会话数据采用覆盖式写入策略，以保证本地缓存状态与服务端数据保持一致；消息数据则采用基于时间戳的增量拉取策略，有效避免历史消息的重复下载，降低网络请求开销。

所有同步后的数据统一持久化至本地 SQLite 数据库，UI 层始终以本地数据库作为唯一数据来源进行渲染，从而在网络不稳定或离线场景下依然能够提供完整的历史数据浏览能力。同时，该设计显著提升了应用的冷启动速度与整体交互体验，并增强了客户端在异常情况下的数据恢复能力。

幂等化消息发送与可靠重试机制

为确保消息发送过程的可靠性与一致性，系统在客户端侧实现了幂等化的消息发送机制。

在发送文本或媒体消息前，客户端会基于当前时间戳与随机数生成一个全局唯一的消息标识（Message ID），并将该标识随消息内容一同提交至服务端。服务端可利用该标识对重复请求进行识别与去重，从而保证消息处理的幂等性。

在客户端内部，消息发送过程采用状态机模型进行管理，消息状态在“Sending”、“Sent”与“Failed”之间流转。所有消息在发送前即写入本地数据库，并在网络请求返回后更新其最终状态。该机制使得系统在网络中断、请求超时或应用重启等场景下仍能够安全地执行消息重试与状态恢复，避免消息丢失或重复发送。

可扩展的媒体消息处理管线

在聊天业务中,系统通过在 ChatService 中引入 ImagePicker 与 FileService,构建了一条可扩展的媒体消息处理管线。

该管线对图片、表情及文件等媒体资源的选择、读取、上传与本地存储过程进行了统一封装,使媒体处理逻辑与 UI 展示逻辑解耦。当前实现已支持图片与表情消息的发送,后续仅需在现有管线基础上扩展新的媒体类型解析与上传策略,即可无缝支持语音、视频或其他富媒体消息。

统一状态分发与响应式 UI 更新机制

系统采用 MultiProvider 对认证状态与聊天状态进行统一托管,并在应用入口处完成依赖注入。

UI 层通过 Consumer 或 Selector 订阅相关状态对象,当用户登录状态、联系人列表或会话数据发生变化时,界面能够自动触发重建与刷新,无需手动维护复杂的状态同步逻辑。该响应式状态分发机制有效避免了状态穿透与冗余刷新问题,使得界面层代码更加简洁、清晰,并提升了整体的可维护性与稳定性。

5 开发日志

11.19

在本地无状态程序下，增加登录界面与登录验证程序。由于此时未设计注册逻辑，因此登录与内存中特定的存储模拟用户信息的 map 进行比对实现验证。

待完成逻辑：注册程序

11.20

构建表情发送逻辑，将从网上下载的 3175 张 emoji 资源导入到工程内 *assets/emojis/* 目录下，修改 *_buildEmojiPicker()* 表情选择器下 emoji-Paths 路径的正确生成，并且修正 *_buildEmojiMessage()* 消息气泡中表情的发送逻辑。

11.21

增加个人主页页面，允许修改头像，名称（uid 固定不允许修改，作为该用户的唯一标识符）。

添加底部导航栏，允许选择三个界面，即聊天（聊天列表），通讯录（好友），我（个人中心）。起初设计了一个 widget 直接实现，将该 widget 显示在三个界面各自的底部位置，然而由于混用自动 UI 状态转换和手动导航 *Naviagte pushReplacement*，导致页面对用户的身份信息认知出现 bug，手动切换页面后可能会导致不能读取到当前信息。后将 widget 拓展为 *homeshell* 主页，从而实现自动检测切换。

之前的程序中并没有用户真实的逻辑，修改用户模型后，使登录与用户强相关，允许维护用户自定义设置，如头像，昵称。

12.4

进一步修改用户模型，在本地数据库中依据唯一主键 *user.id* 查询 *users* 表，索引到该用户对应条目。此前登录信息并没有与数据库真实信息相关，因此对于个人用户信息的修改并没有上传到数据库，仅仅应用在持久化登录状态中，我们应用 *update* 将改动上传到数据库。目前阶段仍然没有完成注册逻辑，在数据库 *users* 表中手动插入若干条 *user* 用于测试。

增设联系人 *contacts* 表，并与 *uses* 表建立外键连接。新增了联系人页面，允许用户添加已注册用户为好友，并在通讯录中显示好友的头像，id，昵称。*contacts* 表中存有 *status* 字段，可用于后序好友添加的审批逻辑，目

前均默认为对方直接同意并添加成功。

增加导出数据库选项，以供直接观测到数据条目，然而并没有成功。后面发现是由于忘记开模拟器的 root 选项，这一步访问总是被拒绝。

12.6

丰富个人主页显示内容，将对话数量与联系人数量加入到主页中

新增对话需要通过已添加的联系人实现，新增对话时弹出新的页面，在该页面选取联系人。同时将对话列表中对话的头像显示为对方的头像

12.7

设计了客户端的数据库结构，同样基于 SQLite 实现，与本地数据库建立同样的数据表。

增加了注册逻辑以及密码加密，为了方便测试暂时不采用

目前对话列表直接读取本地 chats 表中所有 chat，下一步新建对话需选取联系人，并且刷新对话表时，也要根据 id 刷新和该用户相关的对话表

12.8

增加服务端注册/登录/查询等短连接请求，使用 HTTP 实现，端口 8080。

目前 users 表固定在服务端，客户端不再需要 users 表，或仅需要保存近期登录成功的少量用户。

由于现在程序开始由本地模式迁移向 CS 模式，我们并不想对原来已有的代码做出过多修改，因此原来插入到本地数据库的逻辑并没有改变，大多数情况下都更改为向服务端发送该条消息即对应请求，收到回复后再导入本地数据库。

下一步：定义消息的收发，实现多个用户之间的消息推送。注册与登录逻辑在当前时刻等待网络连接即可，而我们并不知道消息何时推送，因此需要长时间监听信道，拟采用不同的实现方式。

12.9

目前发送消息涉及到对发送者身份的认证问题，我们不能简单的从消息的 body 中解析出声称的发送者，就认为是实际的发送者。采用 JWT (JSON Web Token) 在服务器端保存身份信息，一种基于 JSON 的、自包含的“令牌”格式，用来在客户端与服务器之间安全地携带身份与声明信息。

增加同步逻辑，登录完成后，从服务器拉取联系人表。

12.10

更改添加联系人逻辑，向服务器查询，查询到后在本地与服务端 contacts 表中同步添加。

在原来的数据库服务中，至多传入 current.id 用于身份验证，大多时候都是无状态的。在添加服务器以后，用户对服务器数据的各种拉取操作都必须用到自己的 token，因此给 auth service 添加 instance 构造，在不注入 database service 的情况下完成调用。

完成更改用户名上传服务器逻辑

更改头像：头像字段原来存储本地的路径，现在将图片上传至服务器，服务器保存文件并返回可访问的 URL，客户端变更为存 URL (users.avatar) 图片在 screen 中的读取方式需要由 file 改为 URL，不能再验证 hasAvatarFile，而要验证 hasAvatarUrl。

Bug：更改头像以后，本地个人中心不会立即刷新，重新登录才会有效 -> 头像 URL 没变，而该路径下的内容发生了变更。由于 Flutter 会缓存图片，导致本次更新看不到，重登/重启才清缓存。我们给返回的 URL 加如时间戳?ts=... 用来避缓存

在本地模式时，我们建立的各个表中存在大量外键约束，这其实建立在我们正确的操作顺序上。而本地在同步服务端数据时，操作顺序变得混乱，可能碰到外键查找失败的情况，比如在添加 contact 之前添加 user 就会失败。我们在服务端数据库做合适的外键约束就足够了，本地数据库作为副本，可以省略外键约束，也必须省略。

用户表仍然有用，用于存储 contact 的用户信息，获取联系人以及添加联系人时都要更新

下一步：更新 chat_list 相关逻辑，更改 chatservice 服务。事实上，新建一个对话和添加一个新联系人的逻辑基本一致，不涉及状态的实时推送，我们都可以复用之前先传服务端，再更新本地数据库的逻辑。

12.11

更新 chat list 相关显示以及逻辑，包括添加逻辑，加载逻辑（重新改为无需 id，对于其他用户的 chat 现在不可能读到），删除逻辑

完成发送消息逻辑，和之前的思路一致，刷新使用本地数据库，每条消息成功发送到服务器后，本地再更新

问题：当前的 chat 列表并没有及时从服务器获取 -> 将所有消息上传到服务器（文本，图片，表情）

问题：本地的 message 表如何更新？目前登录以后就为空 -> 此时，users 作为公共资源允许无限存储，chats, contacts 在获取 token 后，完全删除原来的库，从服务器重新拉取，messages 和 users 存在外键约束，所以删除所有会话时会级联删除该会话下的所有消息，如果后续没有从服务器重新拉取并写回消息，messages 表就会为空。-> 去除外键约束，message 也应该和 chat 一样，在登录后完全接收，再根据 chat id 显示，所有和该用户相关的 chat，依据 id 拉取所有 message 到本地库

问题：消息气泡的显示逻辑目前有误，没办法正确显示左右位置和头像
12.12

处理同时连接两台模拟器

为了正确显示消息列表中最后一条消息的 [表情], [图片], 我们需要在消息发送完成后对数据库中 chat 的 lastmessage 字段做修改。显示中文会生成乱码，我们统一改为显示英文 [emoji], [picture]

增加一个同步 message 的路由，在登录后刷新，和其他刷新逻辑一起放进 authService 中的同步逻辑。

实现实时聊天机制。不使用一直使用的 http 做短时间轮询实现，而是 chatService 中增加 Server-Sent Events (SSE) 为聊天消息提供实时推送。客户端通过 ‘GET /api/stream?chatId=<chatId>’ 打开 SSE 连接并保持不断开。服务器在内存里维护一个订阅列表（用 ‘mutex’ 保护）。当 ‘/api/messages/send’ 插入新消息后，向关联的订阅者广播 JSON 文本。客户端收到事件后把消息写入本地 DB / 状态树，UI 自动刷新。

问题：SSE 连接失败->chatService 在 main 函数中直接启动，早于登录返回 token（本地模式代码的弊病），SSE 在用户还没登录、token 还没拿到的时候就启动了，直接被 abort 掉了 -> 把 ChatService.initialize() 放在了“登录之前”，而 SSE 依赖登录态（token）

问题：自己发的消息显示两次，别人看只发一次 -> 本地发送成功后，插入本地数据库，同一条消息又通过 SSE 从服务器推回来了，数据库自动有去重，本地 cache 不存在 -> 发送消息逻辑更改为只负责发给服务器，删掉 chat-service 中的消息落地本地逻辑，等服务器 SSE 推回来

修复聊天列表中对话的显示 UI, 一对一聊天中, 根据当前用户, 动态显示对方的名字和头像。重新将 ChatListItem 设为无状态组件 StatelessWidget, 不再要求传入 contact, 这可能会导致后添加的好友的头像不能被 chat list screen 正确识别, 因为我们一进入程序就是 chat list screen, 它内部的所有参数都已经固定, 添加完好友以后并不能及时的传入到这个文件的 contacts 中。在服务器存储上, 名字我们依然用 a 与 b 的聊天, 其中 a 是建立对话的那个人, 但是在本地的显示上, ChatListItem 获取该 chat 的另一位用户, 将名称与头像显示为对方。

12.13

增加服务器密码加密 (注册, 登录), 不使用对称加密, 这可能导致密钥泄露 = 所有密码可解, 不符合安全规范。使用限制每秒尝试次数, 带 salt 的 bcrypt。

问题: 服务器极易崩溃 -> SSE 对象没有被及时清除, 即一个账户退出登录了, 它的 SSE 仍存在服务器表中, 程序把 “只在回调里短暂存在的对象地址”, 当成了 “可以长期保存和跨线程使用的连接句柄” -> 优化 SSE 相关逻辑

问题: user 永远增量增加补全, 导致本地已经缓存的 user 信息一直是老的, 该用户如果修改自己的信息不能及时刷新缓存信息 -> 不做增量补全, 全部覆写。更好的逻辑应该在服务端将这种不常发生的特殊事件推送给相关用户。